AI INNOVATION APPLICATION CHALLENGE

Nexus Decision Arena

AI 决策评审与质询沙盘
技术文档

让多个具有不同立场的 AI 专家独立分析、交叉质询、暴露冲突，并由人类完成可追溯的最终裁决。

交付版本
v1.1 · 2026-11-04

证据基线
Git commit 2f91a71（正文）
演进说明见 P03

公开仓库
github.com/LOST-123456/nexus-decision-arena

镜像仓库
gitee.com/lost666666/nexus-decision-arena

EXECUTIVE SUMMARY

作品摘要

Nexus Decision Arena 不是通用 Agent 工作流编辑器，而是一个受约束的高价值决策评审引擎。 它围绕“独立分析 → 交叉质询 → 冲突检测 → 人工裁决 → 可回放报告”建立闭环。

5
一级 Agent
职责与立场明确

3
核心创新
结构化分歧 / 质询 / 决策轨迹

157
单元与集成测试
2f91a71 基线

80/80
重复 Playwright 用例
桌面 + 移动

问题

普通多 Agent 系统常把不同角色的自由文本回答并排展示，缺少可比较的断言、定向质询、未解决分歧的 结构化记录和结论变化的证据链。评委或使用者只能看到“很多观点”，无法快速判断哪些结论值得相信。

方案

系统把 Agent 输出约束为 Claim、Evidence、Challenge、Conflict 等结构化领域对象。Planner、 质询分配、状态迁移和冲突规则由确定性代码执行；LLM 只在受控边界内生成领域对象和语义判断。

验证

工程证据

- ✓ TypeScript strict 类型检查通过
- ✓ 157/157 单元与集成测试通过
- ✓ 桌面与移动 Playwright 4/4 通过
- ✓ 重复稳定性 80/80 用例通过
- ✓ Next.js 15.5.25 生产构建通过

过程证据

- ✓ 40 个细粒度 Git 提交，非一次性提交
- ✓ 140 条去噪后的实质 AI 对话消息
- ✓ 3 条完整核心 Prompt 链
- ✓ Prompt、测试、Commit 相互映射
- ✓ Mock Mode 无外网完成固定流程

文档边界： 本文件只引用仓库中实际存在的测试、输出、截图和提交。尚未执行的人工基线与真实模型规模化评测不会被包装为已完成数据。

BASELINE & EVOLUTION

本文档基线与后续演进

正文的技术叙述、性能数字与用例计数对应证据基线 `2f91a71`。此后仓库继续演进， 本页说明哪些内容仍然成立、哪些规模已经变化——避免读者把正文的 80 个用例读成当前规模。

基线之后新增的能力

安全与可运维

- ✓ 公开部署配置 fail-closed，不安全配置拒绝启动
- ✓ 登录限流（账户 + 来源地址双维）、登录审计
- ✓ Cookie 写入强制 CSRF / Origin 校验
- ✓ Agent 运行中心：取消、重试血缘（事务化）、失败诊断

研究能力

- ✓ 研究质量评分引擎与证据缺口诊断
- ✓ 可执行研究计划、来源监控与影响分析
- ✓ 跨研究资料库与证据复用
- ✓ 报告快照、版本比较与多语言忠实度核查

当前实测规模（正文数字对应基线，本节对应最新提交）

项目	基线 2f91a71	当前
核心单元测试	见正文	847 通过
核心集成测试（真库）	见正文	282 通过
数据层集成测试（真库）	见正文	195 通过
Playwright 用例	80/80（重复运行）	76 用例 × 2 视口，0 失败
Git 提交	40	101（基线之后 61）

v1.1 修正（本轮）

- ✓ **错误出口统一**：5xx 不再回传驱动错误码（`ECONNREFUSED` / SQLSTATE）， 数据库不可用时单独回 `503 database_unavailable`，页面给出可执行的恢复说明
- ✓ **凭据外置**：登录页不再打印口令；口令改为随机值，轮换走 `scripts/rotate-password.mjs`；E2E 从环境变量或本地文件读取
- ✓ **账号清理**：删除 329 个 E2E 泄漏账号（331 → 2）
- ✓ **E2E 可复现性**：容器名按端口自动推导、测试与被测服务固定同一数据库， `agent-retry` 由 1/4 变为 4/4
- ✓ **窄屏溢出**：长研究问题不再撑破 390px 视口（`407 → 390`）， 此前被 `overflow-x: hidden` 掩盖为「看不到滚动条」
- ✓ **运行期守护**：启动器每 60 秒探测数据库，掉线写日志并尝试拉起

CONTENTS & EVIDENCE BASELINE

目录与证据基线

00 本文档基线与后续演进 P03

01 问题与作品定义 P04

02 五角色决策模型与核心对象 P06

03 Cross Examination 协议 P09

04 系统架构、状态机与事件模型 P11

05 意图控制策略与 AI 工程约束 P14

06 可复现性、降级与恢复设计 P17

07 实验验证、稳定性与结论 P19

08 AI 协同证据、视频与投稿清单 P22

证据基线

类别	基线或文件	用途
代码与提交	`main / feature/decision-arena`	完整历史、可访问性、分支复核
设计冻结	`docs/superpowers/specs/2026-09-10-decision-arena-design.md`	产品边界、协议和数据契约
架构说明	`docs/architecture/system-overview.md`	组件、事件、数据库和 correlationId 规则
复现指南	`README.md`	10 分钟环境配置与启动路径
AI 对话导出	`docs/ai-history/0016-codex-conversation-export.md`	可读的对话快照
结构化导出	`docs/ai-history/exports/codex-session-curated.jsonl`	140 条去噪消息的机器可读记录
核心 Prompt 链	`docs/ai-history/0017-core-prompt-chains.md`	三次核心迭代的纠偏与修复过程
稳定性原始输出	`docs/experiments/stability-output.txt`	80/80 Playwright 用例原始结果

证据链关系



AI 历史导出使用仓库内的 `scripts/export-codex-history.ps1` 生成。原始 Codex Desktop 会话 JSONL 的 SHA-256 快照记录为 `ba613a4c56659bea27c52f6d08cdfa43a24264e4376e48a37743f7e4821258d3`。

01 • PRODUCT DEFINITION

问题与作品定义

1.1 真正待解决的问题

决策评审的困难不等于“缺少观点”，而在于观点之间缺少可核验的比较关系。一个可用的 AI 评审系统必须能够回答：

结论层

- 当前建议是什么?
- 结论是否发生变化?
- 变化由谁触发?

证据层

- 关键断言有何证据?
- 哪些内容只是假设?
- 证据是否过期或未核验?

分歧层

- 谁不同意，具体质疑什么?
- 质疑是否已经回应?
- 冲突的严重度与影响范围?

裁决层

- 人类在哪一步介入?
- 采纳或拒绝的依据是什么?
- 能否回放到关键事件?

1.2 产品定义

> **Nexus Decision Arena**：面向真实决策评审的受约束多 Agent 系统。五个 AI 角色先独立分析， 再针对关键 Claim 进行交叉质询；系统把未解决分歧结构化为 Conflict，并交给人类裁决，最终输出可追溯、 可回放、可复现的决策报告。

1.3 与通用 Agent 工作流的差异

维度	常见多 Agent 展示	Nexus 的约束
核心对象	节点、边、自由文本消息	Claim、Evidence、Challenge、Conflict
观点关系	并列展示，依赖阅读者归纳	结构化关系与定向质询
流程控制	节点自由拼装	固定状态机和确定性规则
结论产生	模型直接输出最终答案	系统建议，人类最终裁决
可复现单位	代码或一次输出	完整决策过程链

1.4 明确不做

首版不实现通用工作流画布、不开放无限节点编排、不自动完成高风险决策、不把业务规则写入 UI， 也不要求真实模型 API 才能运行固定 Demo。产品边界由设计冻结规格和测试共同约束。

01 • PRODUCT DEFINITION

固定 Demo：高校实验室安全平台立项评审

2.1 输入

项目方案

高校实验室 AI 危化品库存与安全预警平台，包含二维码盘点、图像识别、补货预测、过期试剂预警和事故风险提示。

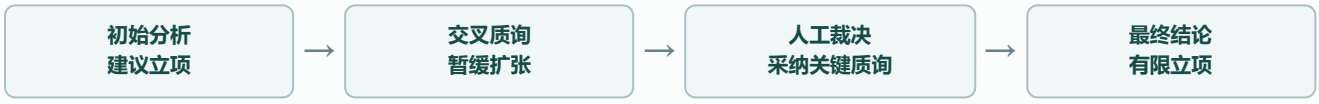
商业目标

每间实验室每年 8 万元；24 个月覆盖 200 间实验室；取得 1600 万元 ARR；毛利率 65%；6 个月完成 MVP。

2.2 四个高风险假设

假设	质询方向	可能导致的变化
24 个月获得 200 间实验室	获客能力、采购周期、试点转化	从规模化扩张转为有限验证
6 个月完成硬件与算法集成	硬件依赖、数据采集、部署条件	追加技术验证门槛
毛利率达到 65%	硬件成本、实施成本、售后服务	要求分阶段核算单位经济
高校采购周期不超过 90 天	预算、招标、验收与合规流程	降低首年收入预期

2.3 预期决策变化



最终建议为先在 3 间实验室验证 12 个月，设置阶段验收门槛；验证通过后再讨论规模化扩张。 结论变化不是模型随机改写，而是由 Challenge、Conflict 和 HumanDecision 共同驱动。

固定 Demo 验收下限

- ✓ 至少 3 个方向不一致的 Claim
- ✓ 至少 5 个结构化 Challenge
- ✓ 至少 3 个 Conflict
- ✓ 展示三种 Human Checkpoint 动作
- ✓ Replay 可回到结论变化时点

实际 fixture 运行证据

- ✓ 12 个 Claims
- ✓ 9 个 opposing Claims
- ✓ 5 个 Challenges
- ✓ 4 个 Conflicts
- ✓ 36 个 ExecutionEvents

真实性边界： 浏览器固定 Demo 明确标注 `DEMO FIXTURE`。它用于稳定演示与测评，不伪装成真实模型调用； 固定 Demo 始终使用离线 fixture；登录后的新评审在界面层要求真实模型 Provider。自动化 E2E 单独使用 Mock AgentRunner，确保浏览器回归与模型网络解耦。

02 • DECISION MODEL

五角色决策模型

系统保留五个一级 Agent，避免角色膨胀和概念堆叠。数据分析、运营、合规、伦理和逻辑分析作为角色内部 Lens 存在；运行、状态展示和推理记录仍以五个一级角色为单位。

市场分析师	技术专家	财务分析师	风险审查员	质询主持人
市场趋势、用户需求、规模口径、数据可信度与获客路径。	架构可行性、实现路径、工程量、依赖条件与部署风险。	成本、收入、现金流、单位经济与运营效率。	合规、隐私、安全、社会、环境、伦理与假设失效。	证据充分性、逻辑一致性、矛盾识别和重新评估。

1.1 AgentRole 契约

```
AgentRole {
  id, name, role, goal, perspective,
  evaluationCriteria: string[],
  evidenceRequired: string[],
  conflictPreference: string[],
  lenses: string[],
  promptVersionId
}
```

1.2 确定性组件

组件	职责	为什么不由 LLM 主导
Planner	选择角色、生成任务、建立执行顺序	保证同输入同顺序和可测试性
Conflict Detector	比较立场、证据、版本和置信度	规则可审计，避免模型随意制造冲突
Report Generator	汇总已校验对象和人工裁决	报告只描述已发生的事实

1.3 角色约束

- 初始分析阶段不得读取其他 Agent 的 Claim，避免角色相互污染。
- 质询者不得质询自己创建的 Claim。
- 主持人只能验证回应是否符合协议，不能擅自修改他方立场。
- 每个断言必须标记类型、立场、重要度、置信度和证据引用。
- 任务生成与角色分配由代码执行，LLM 只输出受 Schema 约束的内容。

> **设计选择：** 一级角色保持五个，内部 Lens 可以多视角展开。这样既保留数据分析、运营和伦理等细分能力，又避免 Decision Map 被十多个角色切碎，降低演示成本和评审理解成本。

02 • DECISION MODEL

核心对象：Claim 与 Evidence

2.1 Claim：可质询的原子断言

```
Claim {
  id: UUIDv7, sessionId, agentRunId, roleId, lens,
  statement,
  type: fact | assumption | prediction | recommendation,
  stance: support | oppose | neutral,
  importance: 1..5, confidence: 0..1,
  evidenceIds: string[],
  status: proposed | supported | contested | accepted | rejected,
  rootClaimId, revisionOfClaimId?, revision,
  relations: Array<{ targetClaimId, type }>,
  respondsToChallengeId?, disposition?
}
```

约束	实现含义
UUIDv7	全球唯一，同时具有时间排序特征；非 UUIDv7 会被 Schema 拒绝。
不可原地覆盖	实质修改创建新 Claim，通过 rootClaimId、revisionOfClaimId、revision 追踪版本。
关系有类型	supports、contradicts、qualifies、depends_on，避免无类型 ID 数组。
接受门槛	重要 Claim 进入 accepted 前必须满足证据和高严重度质询约束。

2.2 Evidence：可核验的支撑或反证

```
Evidence {
  id, sessionId, claimId,
  kind: project_input | calculation | external_reference | assumption,
  title, content, description, sourceRef?,
  direction: supports | opposes,
  reliability: 0..1,
  verificationStatus: unverified | verified | rejected,
  validityPeriod?: { from?, to? },
  retrievedAt, createdBy, agentRunId?
}
```

证据规则

- external_reference 必须有 sourceRef。
- assumption 不得伪装为 verified。
- 超出 validityPeriod 的证据不能支撑最终接受。
- 输入中的数字要记录计算口径或来源。

为什么需要这些字段

- description 让引用型证据可读。
- validityPeriod 处理市场、法规和价格时效。
- reliability 与 verificationStatus 分离，避免把“像真的”当作“已核验”。

02 • DECISION MODEL

核心对象：Challenge 与 Conflict

3.1 Challenge：一次定向质疑

```
Challenge {
  id, sessionId, targetClaimId,
  challengerRunId, challengerRoleId,
  type: evidence_gap | logic_flaw | contradiction | feasibility | priority,
  question,
  context: { triggerClaimIds: string[], explanation },
  requiredEvidence: string[],
  severity: 1..5,
  resolutionStrategy:
    provide_evidence | revise_claim | withdraw_claim | human_decision,
  status: open | answered | validating | resolved | unresolved,
  responseClaimId?, correlationId
}
```

3.2 Conflict：系统检测出的结构化分歧

```
Conflict {
  id, sessionId, claimIds, challengeIds,
  type: evidence | logic | assumption | priority,
  summary, severity: 1..5,
  status: detected | human_review | resolved,
  humanDecisionRequired,
  resolutionSuggestion,
  impactScope: {
    analysisAreas: market | technology | finance | risk | operations[],
    stakeholders: string[]
  }
}
```

> **关键边界：** resolutionSuggestion 只代表建议，不能自动关闭 Conflict。严重度 ≥ 4，或影响三个以上分析域时，必须进入人类裁决。人工裁决前，Conflict 不得直接进入 resolved。

3.3 对象之间的关系



Challenge 的回应仍是新版本 Claim，不额外创造“回复消息”对象。这样版本链、证据和质询关系可以在 同一套领域模型中解释。

03 • CROSS EXAMINATION

Cross Examination 协议

3.1 执行顺序

Step 1	五个角色独立分析，禁止互相读取 Claim。
Step 2	通过 Schema 校验、去重，并选最多 5 个高优先级 Claim。
Step 3	为每个 Claim 分配 1 至 2 个异角色质询者。
Step 4	质询者生成带上下文、证据要求、严重度和解决策略的 Challenge。
Step 5	目标 Agent 创建新版 Claim 回应，不覆盖原 Claim。
Step 6	主持人核验回应，标记 resolved 或 unresolved。
Step 7	冲突检测器生成 Conflict，高严重度冲突进入 Human Checkpoint。

3.2 七条刚性规则

1. 初次分析期间，Agent 不得读取他方输出。
2. 一个 Challenge 只能指向一个 Claim。
3. 质询者不得质询自己的 Claim。
4. Challenge 必须包含问题、上下文、证据要求、严重度和策略。
5. 回应必须产生新版本 Claim。
6. 主持人不能擅自修改其他 Agent 的立场。
7. 未解决的高严重度 Conflict 不得自动关闭。

3.3 Claim 选择算法

比较顺序固定，避免结果依赖模型调用顺序或偶然排序：

```
importance DESC
  → evidence coverage ASC
  → confidence ASC
  → assumption > prediction > recommendation > fact
  → UUIDv7 ASC
```

3.4 质询者分配

角色	优先质询
市场分析师	市场假设、需求证据缺口、规模口径
技术专家	技术可行性、依赖条件、实现周期
财务分析师	成本、收入、单位经济与现金流假设
风险审查员	合规、伦理和关键假设失效风险
质询主持人	逻辑矛盾、跨角色冲突和证据充分性

状态流、限流与失败隔离

4.1 Challenge 状态机



4.2 运行限制

限制	值	目的
每轮目标 Claim	最多 5 个	控制 Demo 时长与事件规模
每个 Claim 质询者	最多 2 个	保留对抗，避免重复轰炸
补充质询	最多 1 次	防止无限循环
单次模型调用	45 秒超时	避免长尾请求阻塞
Repair Prompt	最多 1 次	修复结构错误但不无限重试

4.3 错误隔离

AgentRun 失败
记录 AGENT_RUN_FAILED，其他关键 Agent 继续运行。

Challenge 失败
只影响当前质询，不阻断其他 Challenge 分支。

结构错误
先解析、再 Schema 校验，最后执行一次 Repair Prompt。

4.4 并发与不可变性

- 五个 Agent 使用并行执行，但每个运行单元有独立 correlationId。
- 交给依赖的 Claim 使用 structuredClone，避免外部修改原对象。
- CHALLENGE_CREATED 发布不可变快照，避免后续状态变化污染事件。
- 响应 Claim 独立校验版本链、Challenge 关联和 session 归属。
- 补充轮使用 PostgreSQL session 级 advisory lock，防止并发重复执行。

> **设计结果：** 协议不是“让 Agent 自由辩论”，而是一个可测试、可限流、可恢复的有限状态流程。

04 • ARCHITECTURE

系统架构与组件边界

Web	Next.js App Router、React、XYFlow、Decision Map、Inspector、Timeline、Human Checkpoint、Report
Command API	Fastify REST、Idempotency-Key、Session commands、HumanDecision、Report
Core Engine	Workflow state machine、Arena orchestration、Cross Examination、Replay、Conflict detection
Shared Contracts	Zod schemas、领域对象、DTO、状态迁移不变量、UUIDv7
Adapters	MockLlmProvider、OpenAI-compatible Provider、结构化输出校验与修复
Persistence	PostgreSQL 16、Drizzle ORM、聚合 View、Event repository、FinalReport repository
Eventing	ExecutionEvent、事务后 EventBus、SSE replay/live、Last-Event-ID

1.1 目录结构

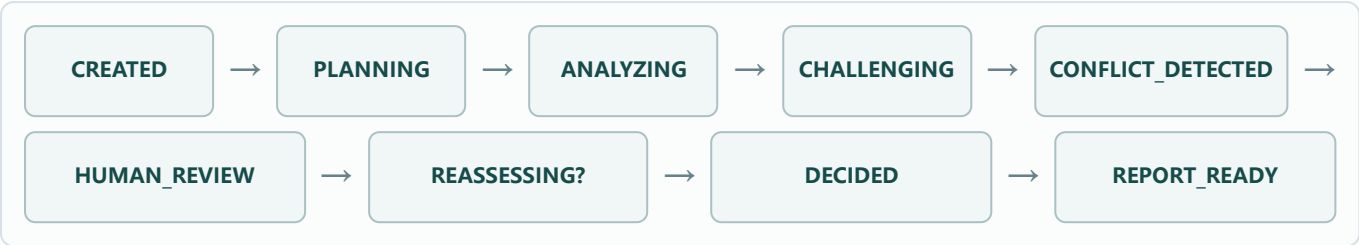
```
apps/web              # Next.js 页面、组件、Decision Map 与浏览器状态
services/core         # Workflow、Arena、API、SSE、Replay、Checkpoint
packages/shared       # 领域 Schema、状态机、DTO、ID 与事件视图
lib/db                # Drizzle Schema、迁移、Repository、SQL View
lib/llm               # Mock 与 OpenAI-compatible Provider
docs/                 # 架构、AI 历史、实验、设计与投稿材料
scripts/              # Demo runner、稳定性测试、AI 历史导出
```

1.2 模块职责

模块	职责	约束
claim-selector	按固定优先级选择 Claim	纯规则，可单元测试
challenger-assigner	分配异角色质询者	不得自问自答
cross-examination	生成质询、回应和核验状态	单分支失败隔离
conflict-detector	根据 Claim / Challenge / 版本规则生成 Conflict	高风险必须转人工
decision-replay	按事件序列恢复地图状态	共享给 Core 与 Web
inspector repository	聚合 Claim、Evidence、Challenge、Conflict	稳定 camelCase DTO

状态机、事件模型与一致性

2.1 DecisionSession 领域状态



operationalStatus 与领域阶段分离：ACTIVE、PAUSED、FAILED、COMPLETED。刷新后通过 phase、 operationalStatus 和事件序列恢复页面，避免把临时前端状态作为事实源。

2.2 ExecutionEvent

```
ExecutionEvent {
  id, sessionId, sequence, correlationId, traceId?,
  type:
    SESSION_STATE_CHANGED |
    AGENT_RUN_STARTED | AGENT_RUN_COMPLETED | AGENT_RUN_FAILED |
    CLAIM_CREATED |
    CHALLENGE_CREATED | CHALLENGE_RESOLVED | CHALLENGE_FAILED |
    CONFLICT_DETECTED | HUMAN_REVIEW_REQUIRED | SESSION_COMPLETED,
  payload, occurredAt, promptVersionId?
}
```

写入顺序

1. 在同一 PostgreSQL 事务中分配 sequence。
2. 追加不可变事件。
3. 事务提交。
4. 提交后才由 EventBus 发布。

SSE 恢复

1. 客户端携带 Last-Event-ID 或 after。
2. 先查询持久化事件进行 replay。
3. 直播事件按 session 连续 sequence 缓冲。
4. 重复或过期事件被幂等忽略。

> **一致性原则：** REST 负责写入命令，SSE 负责单向事实流；任何写操作必须先完成数据库提交，之后才允许前端看到事件。

04 • ARCHITECTURE

数据持久化与 Human-in-the-loop

3.1 核心表

Project	DecisionSession	AgentRole	AgentRun
Claim	Evidence	Challenge	Conflict
HumanDecision	PromptVersion	ExecutionEvent	FinalReport

3.2 Inspector 聚合

```
GET /api/sessions/:sessionId/claims/:claimId/inspector

ClaimInspectorDTO {
  claim,
  evidence[],
  challenges[]: { challenge, responseClaim? },
  conflicts[],
  provenance: { agentRun, promptVersion },
  decisionRationale: {
    outcome, summary, decisiveChallengeIds[],
    evidenceIds[], humanDecisionId?
  }
}
```

SQL View 完成必要 Join 和排序，Repository 映射稳定 DTO。前端不直接依赖数据库表或 View 结构， 减少多次请求并避免 UI 与数据库耦合。

3.3 HumanDecision

```
HumanDecision {
  id, sessionId, conflictId,
  action: accept_challenge | uphold_claim | request_more_analysis,
  rationale,
  affectedClaimIds, affectedAgentRoleIds,
  previousConclusion, newConclusion,
  operatorId, createdAt
}
```

采纳质询

接受异议，推动修订或收敛为有限结论。

维持判断

保留原结论，但必须记录 rationale 和影响。

补充分析

进入一次受控补充轮，然后重新回到人工复核。

> **Human Intent 落地：** 人类选择、理由、受影响 Claim / Agent、前后结论都写入同一事务事件，并可从 Replay 追溯到决策解释。

前端 Human Checkpoint 提供人工理由输入框和影响 Agent 多选。理由与影响范围写入 HumanDecision，并在后续报告和 Replay 中保留。

05 • INTENT CONTROL

意图控制策略：从自然语言到可测试协议

1.1 上下文分层

层	内容	控制目的
系统约束	角色、目标、评价标准、证据要求、冲突偏好、输出 Schema	定义“允许生成什么”
任务上下文	项目输入、当前阶段、目标 Claim、已有 Evidence、允许引用的 ID	定义“本次要判断什么”
修复上下文	失败原因、目标 Schema、错误路径、原输出摘要	定义“如何修正而不扩题”
审计上下文	correlationId、promptVersionId、模型参数摘要、事件 sequence	定义“结果来自哪次调用”

1.2 Prompt 设计原则

约束生成边界

- 输出必须是结构化对象，而不是自然语言长文。
- 枚举值、必填字段、引用关系由 Schema 决定。
- Assumption 必须显式标记，不允许升级为 Fact。

保留人工控制

- LLM 只能建议，不能自动关闭高风险 Conflict。
- 人工纠偏是状态变化的一部分。
- 补充分析最多一次，防止无限扩展。

1.3 用户需求如何被拆成协议

```
“让不同 Agent 互相质疑”
  → Claim 表示可质询断言
  → Challenge 表示一次定向质疑
  → 新版 Claim 表示回应
  → 主持人核验 resolved / unresolved
  → Conflict 表示未解决分歧
  → HumanDecision 表示最终裁决
```

1.4 模型不可控问题的工程方案

JSON parse	解析失败进入明确错误路径，不把文本强行当对象。
Zod validate	校验枚举、必填字段、数值范围、引用和版本关系。
Repair once	只针对结构或引用问题执行一次修复 Prompt。
Isolate	仍失败时记录当前 AgentRun / Challenge 失败，不毒化其他分支。

结构化输出、版本与可追踪链路

2.1 LLM Provider 接口

```
interface LlmProvider {
  generateStructured<T>(
    input: LlmRequest,
    schema: ZodSchema<T>,
    signal: AbortSignal
  ): Promise<LlmResult<T>>
}
```

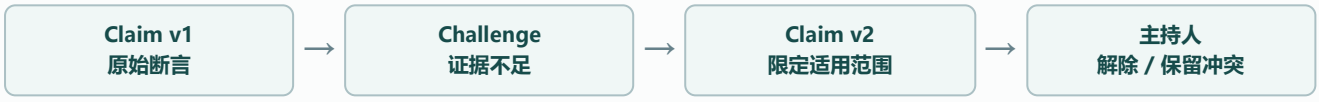
MockLlmProvider

读取固定 fixture，返回确定性结构化对象。无外网、无模型 API 时可完成全流程。

OpenAiCompatibleProvider

通过服务端环境变量配置真实模型，结果经过同一套解析、校验和修复管线。

2.2 Claim 版本不可覆盖



新项目入口会先读取 `/api/runtime`。Mock Mode 只允许浏览固定 Demo； 任意项目必须切换到真实模型模式，运行模式会显示在界面顶部。

旧 Claim 不被改写。rootClaimId 连接版本链，revisionOfClaimId 指向被修订版本，respondsToChallengeId 指向触发修订的质询。Replay 因此可以展示“哪个论点在何时变化”。

2.3 correlationId 传播

工作单元	correlationId	追踪目标
AgentRun	运行级 ID	Prompt、Token、耗时、输出校验、相关 Claim
Challenge	目标 Claim + challenger 的一次尝试	created / resolved / failed 三类事件
HumanDecision	HumanDecision.id	SESSION_STATE_CHANGED 与决策解释

2.4 PromptVersion 与 Commit 的双向映射

- PromptVersion 记录角色或任务使用的提示版本。
- AgentRun 记录 PromptVersion、模型和参数摘要。
- Claim、Challenge、Evidence 保留 agentRunId 或 promptVersionId。
- AI 历史文档记录对应 Commit 和测试证据，形成可复核闭环。

> **意图控制的核心：** 不让 Prompt 决定整个系统结构，而让领域模型、Schema、确定性编排和测试共同约束模型输出。

05 • INTENT CONTROL

AI 协作开发方法与人工纠偏

3.1 协作角色分工

角色	职责	不可让渡的边界
人类负责人	确定产品目标、约束范围、选择方案、最终验收	高风险结论、范围扩张、真实提交
AI 设计协作者	分解需求、比较方案、生成契约、提出风险	不能自行改变比赛目标或声称未验证结果
AI 编程协作者	实现模块、补测试、定位 Bug、整理证据	不能绕过架构规则或隐藏失败
独立复审	从边界、并发、安全和一致性重新检查	发现的问题必须回到测试和 Commit

3.2 一轮协作的标准动作

1 • 目标	用户描述业务意图与验收边界，不直接要求“写一个大而全系统”。
2 • 方案	AI 给出数据结构、状态流、失败路径与测试计划。
3 • 纠偏	人类接受、拒绝或收敛建议，明确不做什么。
4 • 实现	按模块小步落地，每次变更对应功能与 Commit。
5 • 验证	类型、单元、集成、E2E、稳定性或人工检查留下输出。
6 • 复审	独立复审寻找竞态、幻觉、引用悬空和恢复问题。
7 • 记录	把 Prompt、建议、纠偏、代码、测试和 Commit 写入 AI 历史。

3.3 人工纠偏的实际例子

避免角色膨胀

用户提出增加数据分析、运营、合规、伦理和逻辑分析角色。最终方案保留五个一级 Agent， 其余能力作为 Lens 内聚，避免复杂度失控。

接受独立复审

AI 初版通过了基础测试，但复审发现事件对象可变、SSE 有丢事件窗口、幂等记录可能卡死。 这些风险被逐一转化为失败测试和修复 Commit。

3.4 禁止的协作方式

- 不允许一次性要求“把全部功能都做了”，再用单个 initial commit 伪装演进历史。
- 不允许把用户的纠偏、失败测试或 Bug 修复从 AI 历史中删除。
- 不允许用真实模型口吻描述 Mock fixture 结果。
- 不允许以“AI 说通过”代替实际命令输出。

06 • REPRODUCIBILITY

可复现性论证

可复现的不是单次模型文本，而是从输入、配置、Prompt、事件、领域对象到人工裁决和最终报告的完整决策过程。

1.1 复现链



1.2 复现所需固定项

固定项	仓库证据
项目输入	`fixtures/lab-safety-project.json`
预期报告	`fixtures/expected-final-report.json`
依赖版本	`package.json + pnpm-lock.yaml`
环境变量模板	`.env.example`
数据库迁移	`lib/db/migrations/0001..0004`
模型模式	LLM_MODE=mock 或 openai-compatible
事件事实源	execution_events + ExecutionEvent schema
验证输出	test、E2E、build、stability 原始记录

1.3 两种复现路径

离线固定路径

1. 安装依赖。
2. 运行 `corepack pnpm demo:fixture`。
3. 使用 MockLlmProvider 完整推进到 HUMAN_REVIEW。
4. 无需数据库、外网或模型 API。

完整产品路径

1. Docker 启动 PostgreSQL。
2. 复制环境变量并执行迁移。
3. 启动 Web 与 Core。
4. 通过 REST 创建 UUID 会话，SSE 实时更新。
5. 完成人工裁决并打开持久化报告。

> **10 分钟目标：** 干净机器按 README 完成安装、数据库、迁移和启动。网络受限时使用 Mock Mode， 固定 Demo 与 offline fixture runner 仍可运行。

降级、恢复与失败处理

2.1 降级矩阵

故障	系统行为	用户可见状态
真实模型 API 不可用	可切换 Mock Mode；固定 Demo 独立运行	Provider 模式与 fixture 标识明确
单个 AgentRun 失败	记录 AGENT_RUN_FAILED，其他角色继续	失败节点与失败原因可见
单个 Challenge 失败	记录 CHALLENGE_FAILED，不阻塞其他质询	当前质疑标记失败或未解决
结构输出错误	解析和 Schema 校验，最多一次 Repair	重试中或明确失败
SSE 断线	Last-Event-ID 或 after 补发事件	重连后状态恢复
并发重复写	Idempotency-Key + lease fencing	只应用一次结果

2.2 SSE 恢复算法

Subscribe	先订阅实时事件，缓存 replay 期间到达的事件。
Replay	按 after / Last-Event-ID 查询持久化事件并送达。
Gap	检测缺失 sequence，不丢弃后续乱序事件。
Backoff	gap recovery 使用 25ms 到 1000ms 指数退避。
Idempotence	sequence ≤ lastSequence 的事件被忽略。
Cleanup	close / error 时释放订阅、计时器和 heartbeat。

2.3 幂等与 lease fencing

- 同一 Idempotency-Key 的并发请求只允许一个执行者持有 lease。
- 进程崩溃后，过期 lease 可以被安全回收。
- 旧 lease 不能覆盖或删除新 lease 的完成结果。
- human decision 通过 HTTP 与 SSE 同时到达时只应用一次。

2.4 安全边界

输入与密钥

- API 请求由 Zod 校验。
- 项目输入作为不可信数据处理，不能覆盖系统指令。
- API Key 只存在服务端环境变量。
- 日志不输出密钥或完整敏感输入。

报告完整性

- 导出前检查 Claim、Evidence 引用。
- 检查 Conflict 是否仍处于待裁决状态。
- 只汇总已通过 Schema 的对象。
- 最终报告保留人工裁决与决策解释。

账户与数据治理

- 密码使用 scrypt 加盐哈希，不保存明文密码。
- 会话 Cookie 使用 HMAC 签名、HttpOnly 和 7 天过期时间。
- Owner / Reviewer / Viewer 在服务端执行角色校验。
- 历史记录支持继续评审，Owner 才能删除会话及关联数据。

07 • EVALUATION

实验设计与实际验证结果

1.1 已实际执行的工程验证

验证项	命令	结果
类型检查	`corepack pnpm typecheck`	Shared / Web / DB / LLM / Core 全部通过
角色边界实测	`HTTP owner / viewer smoke`	未登录创建 401；Viewer 创建、启动、删除 403；Owner 删除 204
单元与集成测试	`corepack pnpm test`	157/157 通过
E2E	`corepack pnpm --filter @nexus/web test:e2e`	桌面与移动 4/4 通过
生产构建	`corepack pnpm --filter @nexus/web build`	Next.js 15.5.25 通过
重复稳定性	`corepack pnpm test:stability`	80/80 Playwright 用例通过
离线 fixture	`corepack pnpm demo:fixture`	到达 HUMAN_REVIEW，输出 12 Claims / 5 Challenges / 4 Conflicts / 36 Events

1.2 稳定性输出解释

20×
桌面固定流程重复
每个重复包含 2 个 E2E

20×
移动固定流程重复
每个重复包含 2 个 E2E

80/80
最终测试用例
原始输出已入库

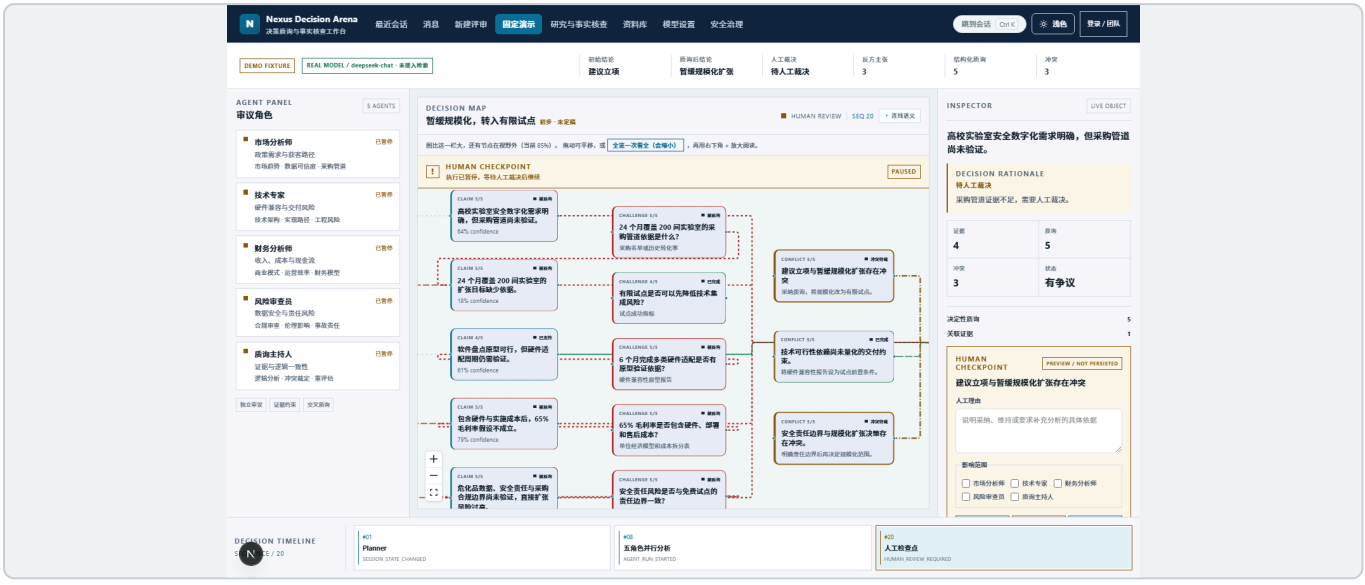
注意分母的准确含义：命令使用 `--repeat-each=20`，项目为桌面与移动，每个项目有 2 个 E2E，因此 2 × 2 × 20 = 80 个 Playwright 测试用例。文档不把 80 个用例误写成 80 个不同业务场景。

1.3 已验证的关键行为

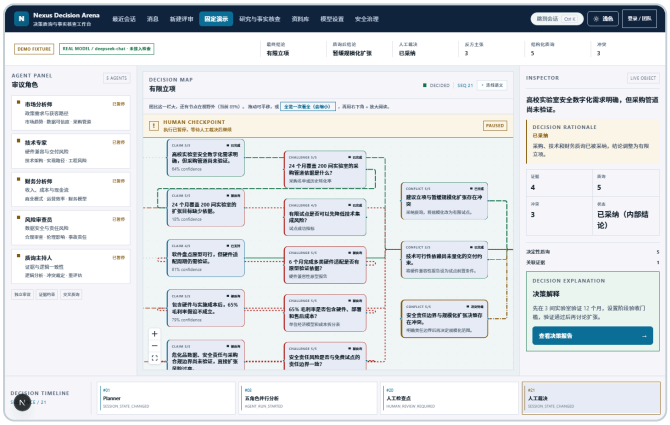
- ✓ 固定 Demo 到达“有限立项”
- ✓ live session 通过 SSE 更新工作区
- ✓ 人工决策后生成持久化报告
- ✓ 移动和桌面关键区域无重叠
- ✓ 补充轮只执行一次
- ✓ 旧 Claim 不会被原地修改
- ✓ 事件重放不丢最终人工裁决
- ✓ 报告只在终态生成

原始输出： `docs/experiments/stability-output.txt` 记录命令、起止时间、退出码、完整 stdout/stderr 和最终 `80 passed`。

产品流程界面证据



桌面 1440×900：Decision Map 展示 Agent、Claim、Challenge 与 Conflict，Inspector 可聚合查看证据和决策解释。



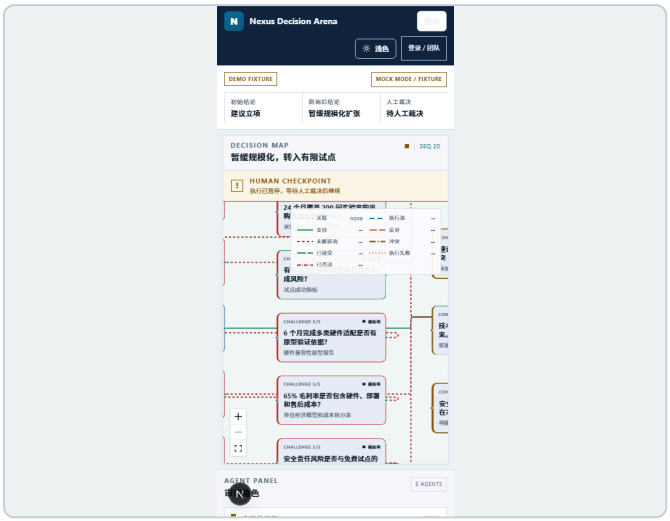
人工裁决后：最终结论切换为“有限立项”，Timeline 保留状态变化。



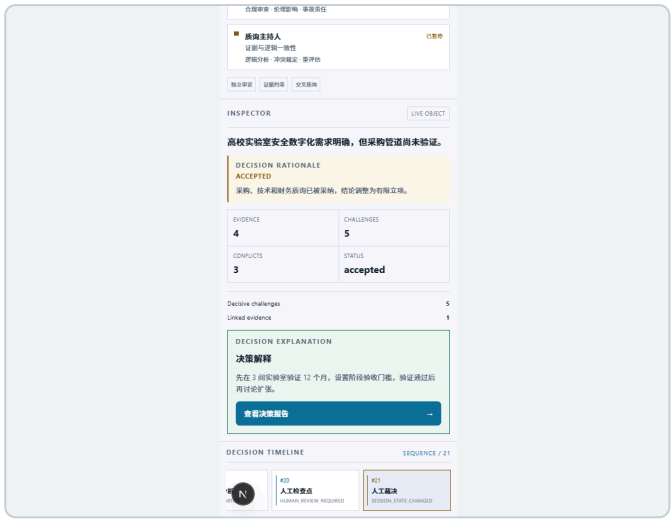
FinalReport：包含 Executive Summary、关键冲突、人工裁决与下一步行动。

截图来源：`artifacts/playwright/`。同一组 Playwright 测试同时验证桌面和移动视口， 并检查关键区域几何重叠与最终结论。

移动端适配与评估边界



移动端 390×844：Decision Map 纵向主线，Inspector 与 Agent 面板通过抽屉组织。



移动端最终结论与决策解释保持可读，未因缩放导致主要操作不可用。

3.1 指标定义

指标	定义	当前状态
评审耗时	从输入提交到出现可审阅结论	实验计划已定义，人工基线与真实模型大规模运行未执行
风险发现数	初始风险与质询后新增风险数量	fixture 计数已记录；统计口径需追加实验 CSV
结论一致性	同输入、同模式下重复运行是否得到一致结论	固定流程稳定性已验证，未宣称为通用模型一致性
人工介入价值	无人工与有人工裁决的结论差异	协议已实现；对照组实验尚未执行

> **不虚构效果：** 本作品当前可证明的是工程可运行、固定流程可复现、关键边界有测试覆盖。它尚未证明在真实组织中可以 缩短一定比例的评审时间或提升一定比例的正确率，因此正文不做这类结论。

3.2 后续可执行实验

- 使用同一项目材料完成人工单角色基线，记录完成时间和风险发现。
- 分别在 Mock Mode 与真实模型模式完成 20 次运行，不混合两组结果。
- 保留 Claim、Challenge、Conflict、HumanDecision 和 Event sequence 原始 ID。
- 对无人工裁决和有人工裁决两条路径使用同一 fixture。
- 失败运行不得从稳定性分母中删除。

实验模板与 CSV 表头见 `docs/experiments/evaluation-plan.md`，所有未填写字段保持空白。

08 • AI COLLABORATION

AI 协同证据：三条完整 Prompt 链

三条链路覆盖数据结构、并发状态和事件恢复三个高风险主题。每条都保留用户 Prompt、AI 建议、人工纠偏、 Bug 修复、回归测试和 Commit。

链路	用户目标	关键纠偏 / Bug	Commit
1. Decision Graph 数据结构	约束 Claim / Evidence / Challenge / Conflict，避免不可维护嵌套 JSON	普通 UUID 校验放宽了 UUIDv7 全局约束	`901ce8c`
2. 并行 Agent 状态竞争	隔离 Agent 和 Challenge 失败，确保 Claim 不可变	依赖可修改原 Claim；事件对象后续可变；补充轮并发重复	`c7b0c8b` `e47c016` `cfad589`
3. SSE 事件丢失与幂等 lease	支持 Last-Event-ID 回放、连续序号和持久化幂等	replay/live 窗口丢事件、乱序误丢、崩溃留下 processing、旧 lease 覆盖新结果	`ac90dbe` `39f8ce7` `c77551a` `79f7e25`

证据文件

可读对话导出

`docs/ai-history/0016-codex-conversation-export.md`

140 条去噪后的实质用户 / AI 消息。

结构化对话快照

`docs/ai-history/exports/codex-session-curated.jsonl`

按消息逐行保存，便于评审检索与溯源。

Prompt 链

`docs/ai-history/0017-core-prompt-chains.md`

三条链的完整问题、建议、纠偏与验证。

导出脚本

`scripts/export-codex-history.ps1`

从 Codex Desktop 原始 JSONL 生成快照。

> **去噪原则：** 只保留与产品、架构、实现、Bug、测试和提交有关的实质消息；过滤环境上下文、工具输出、寒暄、 重复状态和子 Agent 通知。

Prompt 链 1：Decision Graph 与 UUIDv7

1.1 用户 Prompt

```
Claim ID 使用 UUID，避免冲突。
Claim 增加版本号和时间戳，保留变更历史。
Claim 增加 relatedClaimIds，表达逻辑关系。
Evidence 增加 description 和 validityPeriod。
Challenge 增加 context 和 resolutionStrategy。
Conflict 增加 resolutionSuggestion 和 impactScope。
```

1.2 AI 建议

- 所有核心对象统一 UUIDv7，兼顾唯一性和时间排序。
- Claim 不原地修改，每次实质变化产生新版本。
- 用 rootClaimId、revisionOfClaimId、revision 表达版本链。
- 把无类型 relatedClaimIds 改为 supports、contradicts、qualifies、depends_on。
- Conflict 建议不能自动关闭冲突，高严重度必须转人工。

1.3 人工纠偏

用户接受 schema，并补充要求：ExecutionEvent 增加 correlationId；Inspector 增加可追溯决策解释； 报告加入 Decision Explanation；多语言只预留 locale，首版不扩展双语功能。

1.4 发现并修复 Bug

> 最初使用 `z.string().uuid()`，UUID v1/v4 也会通过，无法落实“所有持久化 ID 必须为 UUIDv7”的约束。

修复要求：版本位必须为 7；RFC 变体位必须为 8、9、a 或 b；newId 通过；v4 与非 UUID 拒绝。

```
共享包测试：9/9 通过
TypeScript 类型检查：通过
最终 Commit：901ce8c
设计规格：docs/superpowers/specs/2026-09-10-decision-arena-design.md
```

1.5 视频展示建议

- 同屏展示原始 Prompt 与 AI 回复。
- 切到 `IdSchema` 修复前后的代码 Diff。
- 展示 UUIDv7 通过、v4 拒绝的测试输出。
- 最后显示 Commit `901ce8c` 与设计规格位置。

Prompt 链 2：并行状态竞争与不可变性

2.1 用户 Prompt

```
实现多 Agent 分析和交叉质询。
使用 Promise.allSettled 隔离单个 Agent 失败。
每个 Challenge 必须有独立 try/catch，不能中断其他 Challenge。
原 Claim 不可被原地修改，回应必须形成新版本。
```

2.2 AI 建议

- 五个 Agent 并行执行，每个 AgentRun 使用独立 correlationId。
- 单 Agent 失败发出 AGENT_RUN_FAILED，其他 Agent 继续。
- 每个 Challenge 独立执行，失败发出 CHALLENGE_FAILED。
- 响应 Claim 必须满足版本、Challenge 和 session 关联约束。

2.3 真实 Bug 与人工纠偏

问题	修复
Challenge 依赖可原地修改原 Claim	交付给依赖的 Claim 使用 structuredClone。
响应 Claim 可复用旧 revision	增加独立校验：版本链、Challenge 关联、session 归属。
CHALLENGE_CREATED 事件对象后续仍可变化	发布 structuredClone(challenge) 不可变快照。
补充轮并发请求重复执行	PostgreSQL session advisory lock + 内存等价串行锁。

2.4 回归测试

```
原 Claim 不可修改
无效 revision 被拒绝
Challenge 失败不阻断其他 Challenge
CHALLENGE_CREATED 在后续状态变化后保持 open 快照
并发补充轮只执行一次
补充轮不重复运行 Agent 分析
```

验证结果

- Core 测试通过
- TypeScript 类型检查通过
- 相关 Commit：c7b0c8b / e47c016 / cfad589

视频展示建议

- 展示 structuredClone Diff
- 展示并发测试从失败到通过
- 展示 advisory lock 与单飞断言

Prompt 链 3：SSE 丢失、乱序与幂等 lease

3.1 用户 Prompt

```
使用 SSE 将 ExecutionEvent 推送给前端。
支持 Last-Event-ID 重连和事件回放。
后端事务提交后才能发布事件。
客户端必须能恢复关键状态。
持久化幂等请求不能重复执行。
```

3.2 人工纠偏与四个真实问题

1. replay 结束到开始订阅之间存在丢失事件窗口。
2. replay 与实时事件合并时，低序号乱序事件在 gap 补齐前被错误丢弃。
3. 慢速 replay 期间客户端断开，订阅和 heartbeat 可能未清理。
4. 幂等处理中的进程崩溃可能留下永久 processing 记录。

3.3 修复路线

问题	修复
事件窗口	先订阅并缓存实时事件，再读取并写入 replay。
乱序	expectedSequence 与 pending map，只按连续序号推进。
gap 轮询	25ms 到 1000ms 指数退避，防高频 DB 查询。
连接清理	replay 前注册 close / error 清理。
崩溃卡死	幂等记录增加 lease token 和过期回收。
旧 lease 覆盖	过期 lease 回收时使用 fencing，旧 lease 不能删除新结果。

3.4 回归测试与验证

```
SSE replay 期间发布的事件不丢失
2 / 3 先到的乱序事件最终按 1 / 2 / 3 输出
慢速 replay 断开后订阅和 heartbeat 被释放
gap recovery 有上限退避且无重连风暴
同 key 并发请求只执行一次
过期 lease 可以安全回收，旧 lease 无法覆盖新 lease

Core / DB / TypeScript：通过
Playwright：4/4
稳定性：桌面 + 移动 80/80 用例通过
相关 Commit：ac90dbe / 39f8ce7 / c77551a / 79f7e25
```

3.5 视频展示建议

- 展示原始需求、失败测试和事件序列图。
- 展示乱序事件 2、3 先到时的 pending buffer。
- 展示 lease fencing 测试和 SSE 重连日志。
- 切换到 Commit Hash 与稳定性原始输出。

10 分钟极速复现指南

4.1 环境要求

Node.js 22

仓库锁定 pnpm 10.6.5，使用 corepack 调用。

Docker Desktop

启动 PostgreSQL 16；离线 fixture runner 不依赖数据库。

Git / Chrome

浏览产品；Playwright 需要 Chromium。

4.2 启动步骤

```
corepack pnpm install
docker compose up -d postgres
Copy-Item .env.example .env       # PowerShell
corepack pnpm --filter @nexus/db db:migrate
corepack pnpm dev
```

入口	地址
Web	`http://127.0.0.1:3000`
固定 Demo	`http://127.0.0.1:3000/sessions/demo`
Core API	`http://127.0.0.1:4100`
Health	`http://127.0.0.1:4100/health`
登录与团队	`http://127.0.0.1:3000/login`
历史记录	`http://127.0.0.1:3000/history`
安全治理	`http://127.0.0.1:3000/security`

4.3 两条演示路径

固定 Demo（推荐录屏）

1. 打开 /sessions/demo。
2. 展示初始分析“建议立项”。
3. 展示质询后的“暂缓扩张”和采购渠道冲突。
4. 执行“采纳质询”。
5. 展示最终“有限立项”和决策解释。
6. 点击 Timeline 回放。
7. 打开最终报告。

登录后新建评审（真实模型）

1. 在 /login 使用 Owner 或 Reviewer 登录。
2. POST /api/sessions 与 /start，服务端验证角色与 Idempotency-Key。
3. 打开 /sessions/:id 观察 SSE 更新。
4. 完成 HumanDecision。
5. 打开 /sessions/:id/report，并在 /history 继续评审。

4.4 离线复现

```
corepack pnpm demo:fixture
```

使用 fixtures 与 MockLlmProvider，不需要数据库、外网或模型 API。

4.5 新建自己的评审

打开 `/sessions/new`，填写项目名称、简介、目标用户、商业模式和预期数据， 点击“开始评审”。前端创建 DecisionSession 并调用 start 接口，随后跳转到 `/sessions/:id`，通过 SSE 查看实时分析与人工检查点。

> **常见故障：** PostgreSQL 5432 被占用时修改 compose 与 DATABASE_URL；Playwright 浏览器缺失时安装 Chromium； 页面无事件时先检查 Core /health 和浏览器 SSE 连接。

08 • SUBMISSION

8 分钟视频时间图与录制边界

时间	内容	必须出现的证据
0:00–0:20	初始结论、关键冲突和最终结论概览	产品画面
0:20–0:50	固定项目输入与五角色配置	输入、角色、DEMO FIXTURE 标识
0:50–1:40	五角色独立 Claim 与 Evidence	Agent 状态、证据列表
1:40–2:50	Challenge、回应、主持人评估	结构化质疑与新版 Claim
2:50–3:40	Conflict Map 与高风险假设	冲突严重度、影响范围
3:40–4:30	Human Checkpoint 三种动作	选择、理由、前后结论
4:30–5:20	Replay、决策解释与最终报告	Timeline 与报告
5:20–6:15	AI 案例 1：Decision Graph / UUIDv7	Prompt、Diff、测试、Commit
6:15–7:10	AI 案例 2：并行状态竞争	纠偏、Bug、回归测试、Commit
7:10–8:00	AI 案例 3：Cross Examination / SSE	协议、测试、稳定性、Commit

5.1 画面素材清单

功能录屏

- ✓ 固定 Demo 全流程
- ✓ Inspector 展开证据与版本
- ✓ Human Checkpoint 三种动作
- ✓ Timeline 回放与最终报告

AI 协同录屏

- ✓ 原始 Prompt 文本
- ✓ AI 修改建议
- ✓ 人的纠偏与 Bug 修复
- ✓ 代码 Diff、测试输出、Commit Hash

5.2 当前提交状态

- ✓ 代码仓库、Git 历史、README、AI 对话导出和 Prompt 链已完成。
- ✓ 技术 PDF 可由本 HTML 源文件重复渲染。
- ✓ 已生成 7 分 54 秒、1920×1080、H.264 的自动证据成片。
- ✓ 视频底部包含烧录中文字幕，并提供独立 zh-CN SRT 文件。
- □ 团队实名、学校、指导老师、联系方式和赛事封面模板按平台要求补充。

> **录制真实性：** 视频中的 AI 协同片段应直接展示实际对话或可验证的对话导出，不应使用重新扮演的提示词冒充原始交互。

08 • SUBMISSION

提交索引与最终检查

6.1 公开访问

GitHub

`https://github.com/LOST-123456/nexus-decision-arena`

Gitee

`https://gitee.com/lost666666/nexus-decision-arena`

6.2 核心材料索引

要求	文件 / 地址	状态
公开代码仓库	GitHub + Gitee，main / feature 分支同步	完成
细粒度 Git 历史	47 个提交，首个为设计冻结	完成
AI 对话导出	0016 可读导出 + JSONL 结构化快照	完成
3 条核心 Prompt 链	0017-core-prompt-chains.md	完成
10 分钟复现指南	README.md	完成
环境变量模板	.env.example	完成
技术文档 PDF	本文件渲染输出	可重复生成
演示视频	MP4 1080p、7:54，中文硬字幕 + SRT	已生成
实名与封面	按赛事平台填写	待外部信息

6.3 最终验收清单

产品与工程

- ✓ 新用户无需培训完成一次评审
- ✓ 分析 → 质询 → 冲突 → 裁决 → 报告完整可见
- ✓ 单 Agent 失败不导致会话无响应
- ✓ 结构错误可修复或明确失败
- ✓ 刷新后关键状态可恢复
- ✓ Mock Mode 无外网可完整运行

Vibe Coding 证据

- ✓ Git 历史与功能演进一致
- ✓ Prompt 与 Commit 可映射
- ✓ 至少三条完整协作链
- ✓ 失败测试和修复过程保留
- ✓ 技术文档不超过 30 页
- ✓ 视频总长 5–8 分钟且协同片段不少于 2 分钟

> **文档结束：** 本文档与仓库同源，可由 `scripts/render-technical-pdf.ps1` 在 Chrome 或 Edge 中重复渲染为 A4 PDF。